

1 Composition laws for the intrinsic scale

Source: PrimeTensor/Scale/Laws.lean

What this file establishes

PrimeTensor/Scale.lean built the ladder of closeness relations $\text{ScaleWithin}(\ell, -, -)$ and proved them reflexive and symmetric. It did not prove any statement relating two of them, and closed with the observation that a Cauchy notion on this scale needs one. This file supplies it, in the form the completion will use:

$$\text{ScaleWithin}(\text{succ } \ell, a, b) \wedge \text{ScaleWithin}(\text{succ } \ell, b, c) \longrightarrow \text{ScaleWithin}(\ell, a, c).$$

Two steps, each at one level finer than the target, compose into one step at the target level. This is the multiplicative triangle inequality, and the level bookkeeping in it is not an artefact: by PrimeTensor/Scale.lean the tolerance at level $\text{succ } \ell$ is the square root of the tolerance at level ℓ , so “one level finer” is exactly “half the error” in the multiplicative sense, and two halves make a whole.

Getting there needs three things the previous files left out: that ratios compose, $\text{ratio}(a, c) = \text{ratio}(a, b) \cdot \text{ratio}(b, c)$; that the squaring ladder is multiplicative, $\text{scalePow}(xy, \ell) = \text{scalePow}(x, \ell) \text{scalePow}(y, \ell)$; and the arithmetic core, that $x^2 < 2$ and $y^2 < 2$ force $xy < 2$. The third is where the work is, and it is done on the natural-number cross-products underlying PrimeRatio — the file introduces no real numbers and no square roots.

1.1 Ratios compose

Lemma 1.1 (MulRat.inv_mul). $a^{-1} \cdot a = 1$ for every $a : \text{MulRat}$.

```
@[simp] theorem inv_mul (a : MulRat) : a⁻¹ * a = 1 := by
  rw [mul_comm]
  exact mul_inv a
```

Proof. Commute the product and apply mul_inv of PrimeTensor/MulRat.lean. □

Remark 1.2 (Filling a gap, not proving something new). PrimeTensor/MulRat.lean proved $a \cdot a^{-1} = 1$ and, separately, commutativity, but never stated the left-handed form; since MulRat carries a bare Mul instance and not a group structure, nothing derives it automatically. It is marked `simp` here because the cancellation in Lemma 1.3 is where it is wanted.

Lemma 1.3 (MulRat.ratio_comp). $\text{ratio}(a, c) = \text{ratio}(a, b) \cdot \text{ratio}(b, c)$ for all $a, b, c : \text{MulRat}$.

```
theorem ratio_comp (a b c : MulRat) :
  ratio a c = ratio a b * ratio b c := by
  unfold ratio
  symm
  calc
    (a * b⁻¹) * (b * c⁻¹)
      = ((a * b⁻¹) * b) * c⁻¹ := (mul_assoc (a * b⁻¹) b c⁻¹).symm
  _ = (a * (b⁻¹ * b)) * c⁻¹ := by
    rw [mul_assoc a b⁻¹ b]
```

```

_ = (a * 1) * c⁻¹ := by
  rw [inv_mul b]
_ = a * c⁻¹ := by
  rw [mul_one]

```

Proof. Unfolding ratio of `PrimeTensor/Order.lean`, the claim is $ac^{-1} = (ab^{-1})(bc^{-1})$, and it is proved from right to left: reassociate to expose $b^{-1}b$, cancel it by Lemma 1.1, and remove the unit. The intermediate magnitude b leaves no trace, which is the point — the oriented ratio is a cocycle. $\square$

1.2 The ladder is multiplicative

Lemma 1.4 (`MulRat.mul_mul_shuffle`, private). $(xy)(xy) = (xx)(yy)$ for all $x, y : \text{MulRat}$.

Proof. Six rewrites, alternating associativity and the single commutation $yx = xy$:

$$(xy)(xy) = x(y(xy)) = x((yx)y) = x((xy)y) = (x(xy))y = ((xx)y)y = (xx)(yy).$$

$\square$

Remark 1.5 (Why this is spelled out). The identity is a pure consequence of associativity and commutativity, and on a type carrying a `CommMonoid` instance it would be one call to `ac_rfl` or `ring`. `MulRat` carries only `One`, `Mul` and `Inv` instances, with associativity and commutativity available as named theorems and not as instance data, so the rewriting has to be written out. The contrast is visible inside this same file: Theorem 1.10 does its regrouping on $\mathbb{N}$, where the algebraic instances are in place, and there each regrouping step is a single `ac_rfl`.

Lemma 1.6 (`MulRat.scalePow_mul`). $\text{scalePow}(x \cdot y, \ell) = \text{scalePow}(x, \ell) \cdot \text{scalePow}(y, \ell)$ for all $x, y : \text{MulRat}$ and every level ℓ .

```

theorem scalePow_mul (x y : MulRat) :
  ∀ level : Depth,
    scalePow (x * y) level = scalePow x level * scalePow y level
| .one => rfl
| .succ level => by
  change
    scalePow (x * y) level * scalePow (x * y) level =
      (scalePow x level * scalePow x level) *
      (scalePow y level * scalePow y level)
  rw [scalePow_mul x y level]
  exact mul_mul_shuffle (scalePow x level) (scalePow y level)

```

Proof. By structural recursion on the level. At one both sides are xy , so the claim is `rfl`. At `succ ℓ` both sides unfold to squares; the recursive call rewrites $\text{scalePow}(xy, \ell)$ to $\text{scalePow}(x, \ell) \text{scalePow}(y, \ell)$, and Lemma 1.4 regroupes the square of that product into the product of the squares. $\square$

Read through the exponent formula of `PrimeTensor/Scale.lean`, this is $(xy)^n = x^n y^n$ for $n = 2^{|\ell|-1}$; but it is proved directly on the ladder, so no exponent arithmetic enters the file.

1.3 The ruler as a natural number

Lemma 1.7 (`MulRat.multiset_eval_pos`, private). $0 < q.\text{eval}$ for every $q : \text{PrimeMultiset}$.

Proof. By induction on the quotient, it suffices to treat a bag, and there `PrimeBag.one_le_eval` of `PrimeTensor/Prime.lean` gives $1 \leq \text{eval}$. $\square$

Remark 1.8 (The same lemma, a third time). This statement is proved privately in `PrimeTensor/MulRat.lean`, again in `PrimeTensor/Order.lean` as `multiset_eval_pos_order`, and again here. Each copy is private, so no later file can reach the earlier one; the duplication is the price of that. What it says is the positivity that makes the cross-product order well behaved at all: every barcode denotes a magnitude ≥ 1 , so no denominator can vanish and no comparison degenerates.

Lemma 1.9 (`MulRat.twoRatio_upper_eval` and `twoRatio_lower_eval`). $\text{twoRatio.upper.eval} = 2$ and $\text{twoRatio.lower.eval} = 1$.

```
@[simp] theorem twoRatio_upper_eval : twoRatio.upper.eval = 2 := by
  norm_num [twoRatio, primeTwo, PrimeMultiset.eval_ofBag,
    PrimeBag.eval_factor, PrimeBag.eval_one]

@[simp] theorem twoRatio_lower_eval : twoRatio.lower.eval = 1 := by
  rfl
```

Proof. The numerator is the one-atom barcode carrying `primeTwo`: evaluating through `eval_ofBag` and `eval_factor` gives `primeTwo.value` $\cdot$ 1, which `norm_num` reduces to 2. The denominator is the pivot, whose evaluation is 1 definitionally. $\square$

These two are the hinge of the file. Marked `simp`, they let every comparison against the ruler be normalised into a statement about the literal naturals 2 and 1, after which the remaining work is ordinary arithmetic on $\mathbb{N}$ and nothing about barcodes is needed again.

1.4 The arithmetic core

Theorem 1.10 (`MulRat.mul_lt_two_of_sq_lt_two`). If $x \cdot x < \text{two}$ and $y \cdot y < \text{two}$ in `MulRat`, then $x \cdot y < \text{two}$.

Proof. Both hypotheses are about classes, and $<$ on `MulRat` was defined in `PrimeTensor/Order.lean` by lifting `PrimeRatio.Lt` along the quotient, so choose representatives x_r and y_r and write

$$p = x_r.\text{upper.eval}, \quad q = x_r.\text{lower.eval}, \quad u = y_r.\text{upper.eval}, \quad v = y_r.\text{lower.eval},$$

so that x denotes p/q and y denotes u/v . Products of barcodes multiply componentwise, by `upper_mul` and `lower_mul` of `PrimeTensor/Ratio.lean`, and `PrimeRatio.Lt(a,b)` is the cross-product inequality $a.\text{upper.eval} \cdot b.\text{lower.eval} < b.\text{upper.eval} \cdot a.\text{lower.eval}$. Normalising the ruler with Lemma 1.9 turns the two hypotheses and the goal into statements about naturals:

$$p^2 < 2q^2, \quad u^2 < 2v^2, \quad \text{goal: } pu < 2qv.$$

Squaring is strictly monotone on $\mathbb{N}$, so by `Nat.mul_self_lt_mul_self_iff` it is enough to prove the squared goal $(pu)^2 < (2qv)^2$, and that is a chain of two strict steps:

$$(pu)^2 = p^2 u^2 < (2q^2) u^2 = u^2 (2q^2) < (2v^2)(2q^2) = (2qv)^2.$$

The first strict step multiplies the first hypothesis by u^2 and the second multiplies the second hypothesis by $2q^2$; both need the multiplier to be strictly positive, which is Lemma 1.7 applied to $y_r.\text{upper}$ and to $x_r.\text{lower}$. The three equalities are regroupings of a commutative product, each closed by `ac rfl`. $\square$

Design note 1.11 (Squaring in place of a square root). Stated in the reals the theorem is immediate: $x^2 < 2$ and $y^2 < 2$ give $x, y < \sqrt{2}$, hence $xy < 2$. That argument is unavailable here for the reason `PrimeTensor/Scale.lean` recorded — $\sqrt{2}$ is not an element of `MulRat`, which contains exactly the positive rationals, so the intermediate bound cannot even be written down.

The proof above avoids it by never dividing the problem in half. It multiplies the two hypotheses together instead, obtaining $(xy)^2 < 4$, and then descends from $(xy)^2 < 4 = 2^2$ to $xy < 2$ by strict monotonicity of squaring on $\mathbb{N}$ — a square root taken on the integers, where it is exact, rather than on the magnitudes, where it does not exist. Everything the file needs from irrational tolerances is obtained this way: the tolerances are never named, only their squares, and the squares are rational.

1.5 The composition law

Theorem 1.12 (`MulRat.scaleWithin_comp`). *If $\text{ScaleWithin}(\text{succ } \ell, a, b)$ and $\text{ScaleWithin}(\text{succ } \ell, b, c)$, then $\text{ScaleWithin}(\ell, a, c)$.*

```
theorem scaleWithin_comp {level : Depth} {a b c : MulRat}
  (hab : ScaleWithin (.succ level) a b)
  (hbc : ScaleWithin (.succ level) b c) :
  ScaleWithin level a c := by
```

Proof. By definition $\text{scalePow}(r, \text{succ } \ell)$ is $\text{scalePow}(r, \ell)^2$, so the four conjuncts of the two hypotheses read

$$\text{scalePow}(\text{ratio}(a, b), \ell)^2 < \text{two}, \quad \text{scalePow}(\text{ratio}(b, a), \ell)^2 < \text{two},$$

and the same pair for (b, c) and (c, b) .

For the forward conjunct of the goal, rewrite $\text{ratio}(a, c) = \text{ratio}(a, b) \text{ratio}(b, c)$ by Lemma 1.3 and distribute the ladder over the product by Lemma 1.6, reducing the goal to

$$\text{scalePow}(\text{ratio}(a, b), \ell) \cdot \text{scalePow}(\text{ratio}(b, c), \ell) < \text{two},$$

which is Theorem 1.10 applied to the two forward hypotheses. For the backward conjunct, rewrite instead $\text{ratio}(c, a) = \text{ratio}(c, b) \text{ratio}(b, a)$ — note the reversed order — and apply the theorem to the backward hypothesis for (c, b) and the backward hypothesis for (b, a) . $\square$

Remark 1.13 (The level cost is exactly right). By `PrimeTensor/Scale.lean` the tolerance at level ℓ is $2^{1/2^{|\ell|-1}}$, and at $\text{succ } \ell$ it is $2^{1/2^{|\ell|}}$, the square root of the former. So the theorem says: two ratios each below t have product below t^2 , with t the finer tolerance and t^2 the coarser one. The

inequality is sharp in the sense that no smaller target level would follow — $t \cdot t$ is exactly t^2 , not less — so composing costs one level and cannot cost fewer.

That single-level cost is what makes the ladder usable downstream. A Cauchy sequence for this scale will be indexed by depth, and chaining n steps of a telescoping argument will consume n levels; since the levels are the depths of `PrimeTensor/Depth.lean` and those are unbounded, there is always room to pay.

Remark 1.14 (Monotonicity in the level now follows). `PrimeTensor/Scale.lean` noted that closeness at a finer level ought to imply closeness at a coarser one, and that the argument needed a comparison fact not then available. Theorem 1.10 is that fact, and the implication is now a one-line consequence of Theorem 1.12: take the middle magnitude to be a itself and feed `scaleWithin_refl(succ  $\ell$ ,  $a$ )` as the first hypothesis, so that

$$\text{ScaleWithin}(\text{succ } \ell, a, b) \longrightarrow \text{ScaleWithin}(\ell, a, b).$$

The file does not state this, and nothing below needs it in this form; it is recorded here because it settles the question the previous file left open.

Remark 1.15 (What is still missing). The composition law is proved only in the shape it will be used — `succ  $\ell$` and `succ  $\ell$` into ℓ . There is no general statement composing level m with level n into some level determined by both, and no n -fold version chaining a whole finite path. Nor is `ScaleWithin` shown to be an equivalence relation at any fixed level: it is reflexive and symmetric, but transitivity at a single level is false in general, and the level-shifted law above is what replaces it. A completion built on this scale has to be organised around that shift rather than around an equivalence.

Summary of the correspondence

Lean declaration	Kind	Here
<code>MulRat.inv_mul</code>	simp thm	Lem. 1.1
<code>MulRat.ratio_comp</code>	theorem	Lem. 1.3
<code>MulRat.mul_mul_shuffle</code>	private thm	Lem. 1.4
<code>MulRat.scalePow_mul</code>	theorem	Lem. 1.6
<code>MulRat.multiset_eval_pos</code>	private thm	Lem. 1.7
<code>MulRat.twoRatio_upper_eval</code>	simp thm	Lem. 1.9
<code>MulRat.twoRatio_lower_eval</code>	simp thm	Lem. 1.9
<code>MulRat.mul_lt_two_of_sq_lt_two</code>	theorem	Thm. 1.10
<code>MulRat.scaleWithin_comp</code>	theorem	Thm. 1.12

Every declaration of `PrimeTensor/Scale/Laws.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition. Remarks 1.13, 1.14 and 1.15 are stated for the reader and are not declarations of the file.